

실시간 운영체제 기반 전자제어기 프로그래밍

Contents

실시간 운영체제 기반 전자제어기
프로그래밍

Task 관리

Task 관리

Introduction

□ 본 장의 구성

- FreeRTOS가 각 Task에 처리 시간을 할당하는 방법
 - 특정 시점에 실행할 Task를 선택하는 기준
 - Task의 상대 Priority가 시스템 동작에 미치는 영향
- Task 함수와 하나 이상의 Task Instance 생성
- Task 상태와 상태 전이
- 주기적 처리, Idle Task, Priority 변경과 Task 삭제
- Thread Local Storage와 Scheduling Algorithm

Task 관리

□ 기본 형식

○ FreeRTOS Task는 C 함수로 구현한다.

- 하나의 void 포인터 매개변수를 받고 void를 반환한다.
- 일반적으로 무한 루프 안에서 계속 실행한다.

```
void vATaskFunction( void * pvParameters )
```

□ 중요한 규칙

○ Task 함수는 어떤 경로로도 return해서는 안 된다.

- Task가 더 이상 필요하지 않으면 vTaskDelete()를 호출한다.

○ 하나의 Task 함수 정의로 여러 Task Instance를 만들 수 있다.

- 자동 변수는 각 Instance의 Stack에 별도로 존재한다.
- static 변수는 모든 Instance가 같은 객체를 공유한다.

Task 관리

□ 일반적인 Task 함수의 구조

- 자동 변수와 static 변수의 저장 위치를 구분해야 한다.

```
void vATaskFunction( void * pvParameters )
{
    long lStackVariable = 0;
    static long lStaticVariable = 0;

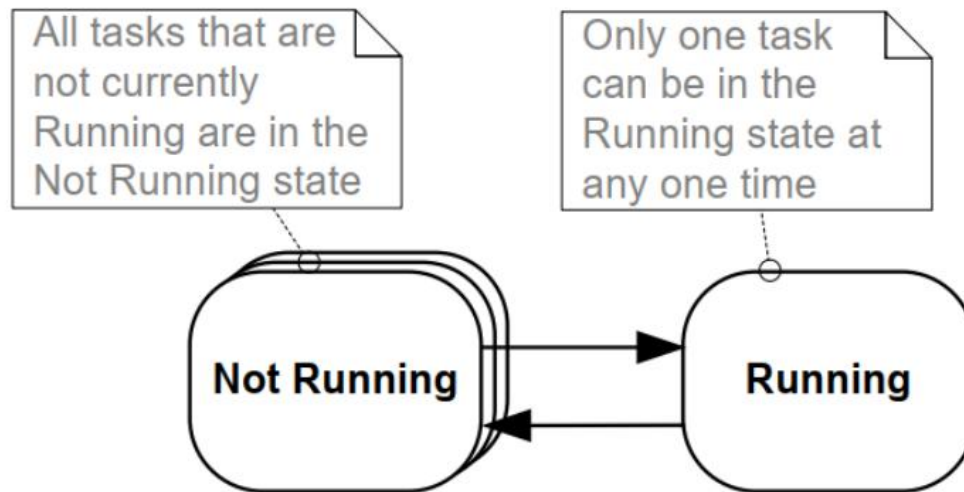
    for( ;; )
    {
        /* Task 기능을 구현하는 코드 */
    }

    /* 무한 루프를 빠져나왔다면 Task를 삭제한다. */
    vTaskDelete( NULL );
}
```

Task 관리

□ Running과 Not Running

- Running Task는 Processor가 실제로 코드를 실행하는 Task이다.
- Not Running Task는 실행 상태를 저장하고 Scheduler의 선택을 기다린다.
- Scheduler만 Task를 Running 상태로 넣거나 뺄 수 있다.
- Running으로 전환되면 이전에 멈춘 지점부터 실행을 계속한다.



Figure

4.1 Top level task states and transitions

- Running과 Not Running의 두 상태에서 시작한다. 이후 Not Running을 Ready, Blocked, Suspended로 세분화한다.
- Not Running 상태의 Task가 Running 상태로 전환되는 것을 switched in 또는 swapped in 이라고 하며 반대 방향은 switched out 또는 swapped out이라고 한다.

Task 관리

□ Task 생성 API

- Static, Restricted, Affinity는 메모리 할당, 권한, 실행 Core 지정 방법의 차이를 나타낸다.

API	특징
xTaskCreate()	가장 단순한 동적 메모리 기반 생성
xTaskCreateStatic()	TCB와 Stack 메모리를 호출자가 제공
xTaskCreateRestricted()	제한된 권한으로 실행되는 Task 생성
xTaskCreateRestrictedStatic()	Restricted Task를 정적 메모리 방식으로 생성
xTaskCreateAffinitySet()	SMP 환경에서 실행 Core 지정
xTaskCreateStaticAffinitySet()	Static Allocation과 Core Affinity를 함께 사용

- 각 Task는 TCB와 Task Stack을 위한 두 개의 RAM 블록을 필요로 한다.

Task 관리

☐ Dynamic Allocation

- 이름에 Static이 없는 API는 실행 중에 System Heap에서 RAM을 할당한다.
- Task Control Block(TCB)와 Task Stack용 Memory를 확보한다.

☐ Static Allocation

- 이름에 Static이 있는 API는 호출자가 미리 할당한 Memory를 전달한다.
- 실행 중 Memory Allocation을 피해야 하는 시스템에 적합하다.

☐ Restricted와 Affinity

- Restricted API는 Memory 접근이 제한된 Task를 만들 수 있는 port에서 사용한다.
 - Restricted가 없는 API로 생성한 Task는 일반적으로 Privileged Mode 로 실행하며 시스템 전체 Memory Map에 접근할 수 있다.
- Affinity API는 SMP를 지원하는 FreeRTOS Port에서 해당 API를 사용하여 Task가 실행될 Core를 지정할 수 있다.

Task 관리

□ 가장 단순한 동적 메모리 기반 Task 생성 함수

```
BaseType_t xTaskCreate(  
    TaskFunction_t pvTaskCode,  
    const char * const pcName,  
    configSTACK_DEPTH_TYPE usStackDepth,  
    void * pvParameters,  
    UBaseType_t uxPriority,  
    TaskHandle_t * pxCreatedTask  
);
```

○ 반환값

- pdPASS : Task를 성공적으로 생성했다.
- pdFAIL : Task를 생성할 Heap Memory가 부족하다.

Task 관리

□ Parameter의 의미

Parameter	설명
pvTaskCode	Task를 구현하는 함수의 포인터
pcName	디버깅에 사용하는 사람이 읽을 수 있는 Task 이름
usStackDepth	Task Stack의 깊이. byte가 아니라 word 단위
pvParameters	Task 함수의 void * 매개변수로 전달할 값
uxPriority	초기 Priority. 최저값은 0, 최고값은 configMAX_PRIORITIES-1
pxCreatedTask	생성된 Task Handle을 저장할 위치. 필요 없으면 NULL

○ Stack 용량은 Stack Depth × Stack Word 폭으로 계산한다.

Task 관리

☐ pcName

- 사람이 읽을 수 있는 디버깅용 Task 이름이다.
- FreeRTOS Kernel 자체는 동작에는 사용하지 않으며 최대 길이는 configMAX_TASK_NAME_LEN이 정한다.

☐ usStackDepth

- Stack 크기를 byte가 아닌 word 단위로 지정한다.
- Stack word가 32 bit이고 값이 128이면 512 byte를 할당한다.

☐ configSTACK_DEPTH_TYPE

- 정의하지 않으면 기본 자료형은 uint16_t이다.
- Stack 용량이 65,535를 넘을 수 있으면 unsigned long이나 size_t와 같은 더 큰 자료형을 사용한다.

Task 관리

□ 동일한 Priority를 가진 두 Task

- 두 Task는 출력 문자열만 다르고 나머지 구조는 동일하다.

```
void vTask1( void * pvParameters )
{
    volatile unsigned long ulCount;
    for( ;; )
    {
        vPrintLine( "Task 1 is running" );
        for( ulCount = 0;
            ulCount < mainDELAY_LOOP_COUNT;
            ulCount++ ) {
            /*단순한 Busy-Wait 지연*/
        }
    }
}
```

```
void vTask2( void * pvParameters )
{
    volatile unsigned long ulCount;
    for( ;; )
    {
        vPrintLine( "Task 2 is running" );
        for( ulCount = 0;
            ulCount < mainDELAY_LOOP_COUNT;
            ulCount++ ) {
            /*단순한 Busy-Wait 지연*/
        }
    }
}
```

- 주기적 출력 효과를 만들기 위해 Null Loop를 임시 지연 방법으로 사용한다.

Task 관리

□ Scheduler 시작 절차

- Task 생성 후 vTaskStartScheduler()를 호출한다.

```
int main( void )
{
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 1, NULL );
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );

    vTaskStartScheduler();

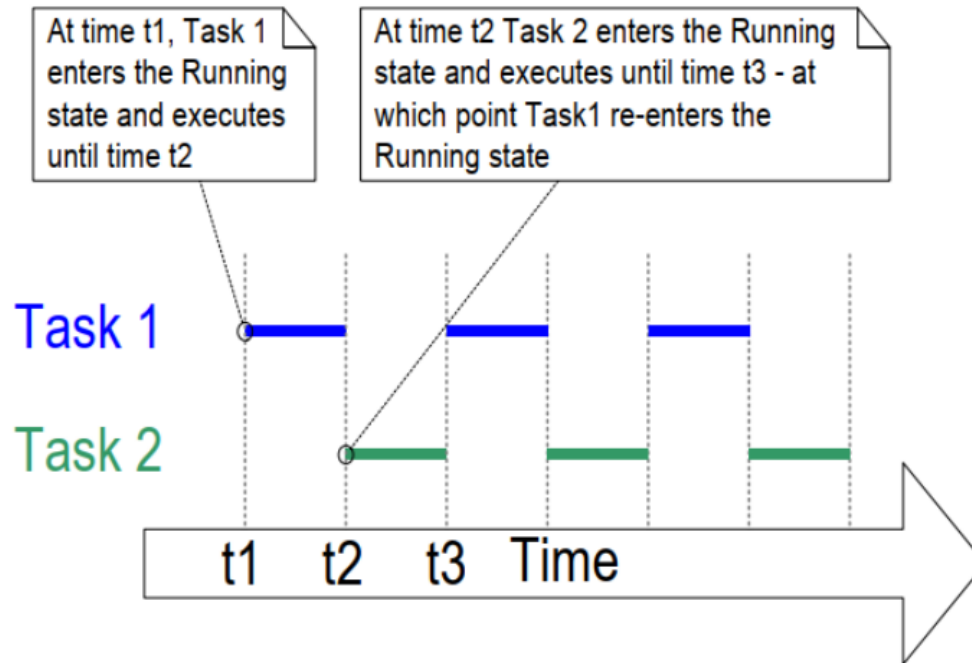
    /* 정상이라면 여기에 도달하지 않는다. */
    for( ;; );
}
```

- main()의 Stack에 선언한 지역 변수는 scheduler 시작 후 Task에서 참조하지 않는다.
- Scheduler가 시작된 뒤 다른 Task 내부에서도 새로운 Task를 생성할 수 있다.

Task 관리

□ 동일 Priority Task의 실행

- 단일 Core에서 동시에 실행하는 것이 아니라 Running 상태를 빠르게 번갈아 사용한다.



Figure

4.3 The actual execution pattern of the two Example 4.1 tasks

- 한 Task가 Running으로 전환되면 다른 Task는 Not Running으로 전환된다.

Task 관리

☐ 단일 Core의 실행

- 두 Task가 동시에 실행되는 것이 아니라 Scheduler의 선택에 따라 번갈아 Running 상태가 된다.
- 한 Task가 Running 상태가 되면 다른 Task는 Not Running 상태로 전환된다.

☐ Windows Simulator의 출력

- 원문의 Simulator에서는 두 Task가 한 번씩 번갈아 출력하는 모습이 뚜렷하다.
- 실제 Embedded Target에서는 출력 함수의 속도에 따라 한 Task가 여러 번 출력한 뒤 교체될 수 있다.

☐ main() Stack 주의

- Scheduler 시작 이후 Task가 main()의 지역 변수를 참조하도록 작성하지 않는다.

Task 관리

□ Scheduler 시작 후 Task 생성

- Task는 main()에서만 생성할 필요가 없다.
- Scheduler가 실행 중이면 다른 Task 내부에서도 생성할 수 있다.

```
void vTask1( void * pvParameters )
{
    volatile unsigned long ul;

    /* Scheduler가 이미 실행 중인 상태에서 Task 2를 생성한다. */
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );

    for( ;; )
    {
        vPrintLine( "Task 1 is running" );
        for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ ) { }
    }
}
```

Task 관리

□ 하나의 Task 함수에서 여러 Instance 생성

○ pvParameters로 서로 다른 문자열을 전달하여 중복 코드를 제거한다.

```
void vTaskFunction( void * pvParameters )
{
    char * pcTaskName;
    volatile unsigned long ul;

    pcTaskName = ( char * ) pvParameters;

    for( ;; )
    {
        vPrintLine( pcTaskName );
        for( ul = 0; ul <
mainDELAY_LOOP_COUNT; ul++ ){}
    }
}
```

```
static const char * pcTextForTask1 = "Task 1 is running";
static const char * pcTextForTask2 = "Task 2 is running";

int main( void )
{
    xTaskCreate( vTaskFunction, "Task 1", 1000,
                ( void * ) pcTextForTask1, 1, NULL );

    xTaskCreate( vTaskFunction, "Task 2", 1000,
                ( void * ) pcTextForTask2, 1, NULL );

    vTaskStartScheduler();
    for(;;){}
}
```

➤ 함수 코드는 공유하지만 각 Task는 독립적인 Stack과 자동 변수를 가진다.

Task 관리

☐ Priority 기본 원칙

- Scheduler는 실행 가능한 Task 중 Priority가 가장 높은 Task를 Running 상태로 선택한다.
- 같은 Priority의 Task가 여러 개이면 실행 기회를 번갈아 가진다.
- 0이 최저이고 configMAX_PRIORITIES-1이 최고이다.

☐ Generic Scheduler

- C로 구현되어 모든 FreeRTOS Architecture Port에서 사용할 수 있다.
- Priority 수가 늘면 RAM 사용량과 Worst-Case Execution Time이 증가할 수 있다.
- configUSE_PORT_OPTIMISED_TASK_SELECTION=0으로 선택한다.

☐ Architecture-Optimized Scheduler

- Architecture-specific Assembly를 사용한다.
- 지원 Architecture에서는 Priority 수와 무관하게 Task 선택 시간이 일정하다.
- 32-bit에서는 최대 32, 64-bit에서는 최대 64 Priority Level을 사용한다.
- 지원 Port는 configUSE_PORT_OPTIMISED_TASK_SELECTION=1을 사용한다.

Task 관리

□ configMAX_PRIORITIES와 Task 선택 시간

구분	Generic	Architecture-Optimized
구현	C	Architecture-specific Assembly
Priority 수	고정 상한 없음	32-bit는 32, 64-bit는 64
Worst-Case 시간	Priority 수가 늘면 길어질 수 있음	Priority 수와 관계없이 일정
설정	configUSE_PORT_OPTIMISED_TASK_SELECTION=0	지원 Port에서 configUSE_PORT_OPTIMISED_TASK_SELECTION=1

Task 관리

☐ Architecture-Optimized 구현

- 지원 Port에서는 configUSE_PORT_OPTIMISED_TASK_SELECTION=1을 사용한다.
- 지원 Port는 설정 상수를 정의하지 않아도 최적화 구현을 기본값으로 사용할 수 있다.
- Priority 수와 관계없이 Task 선택 시간이 일정하다.

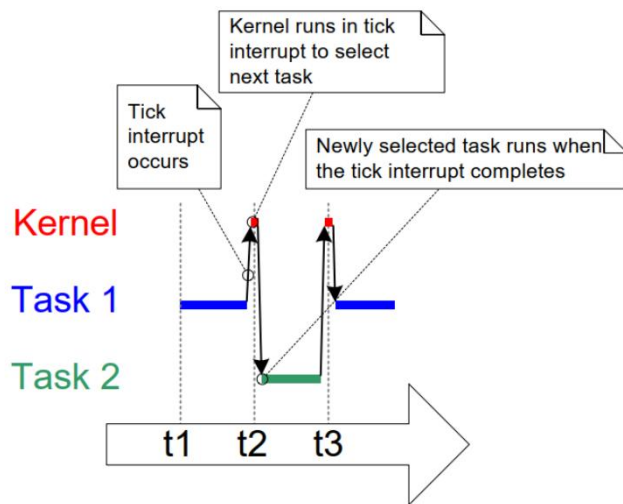
☐ Generic 구현

- configUSE_PORT_OPTIMISED_TASK_SELECTION=0으로 설정한다.
- Priority Level 수를 실용적인 범위로 유지하면 RAM 사용량과 Worst-Case 선택 시간을 줄일 수 있다.
- 이식성이 중요하거나 지원되는 최적화 구현이 없을 때 사용한다.

Task 관리

□ Tick 기반 시간 관리

- Scheduler는 주기적인 Tick Interrupt를 사용하여 다음 Task를 선택한다.
- configTICK_RATE_HZ가 Tick Frequency를 정한다.



○ 설정 예

- configTICK_RATE_HZ = 100 Hz
- Tick Period = 10 ms

○ 시간 변환

- pdMS_TO_TICKS()를 사용하여 ms를 Tick 수로 변환한다.

```
TickType_t xTimeInTicks = pdMS_TO_TICKS( 200 );
```

- Task가 Blocked로 들어가거나 더 높은 Priority Task가 Ready가 되면 Tick 사이에도 Task를 교체할 수 있다.

Task 관리

☐ Time Slice와 Tick Period

- Time Slicing을 사용하면 한 Time Slice는 일반적으로 한 Tick Period와 같다.
- `configTICK_RATE_HZ=1000`이면 Tick Period와 Time Slice는 10 ms이다.

☐ `pdMS_TO_TICKS()`

- 시간을 ms 단위로 표현하면 Tick Frequency가 바뀌어도 응용 코드의 시간 설정을 유지하기 쉽다.
- 원문은 `configTICK_RATE_HZ`가 1000보다 큰 경우 `pdMS_TO_TICKS()`를 사용할 수 없다고 설명한다.

☐ Tick Count

- Scheduler 시작 후 발생한 Tick Interrupt의 누적 횟수이다.
- Overflow가 발생해도 일반적인 지연 처리의 일관성은 Kernel이 관리한다.

Task 관리

□ 서로 다른 Priority로 두 Task 생성

- Task 2는 Priority 2, Task 1은 Priority 1로 생성한다.

```
xTaskCreate( vTaskFunction, "Task 1", 1000, ( void * ) pcTextForTask1, 1, NULL );
```

```
xTaskCreate( vTaskFunction, "Task 2", 1000, ( void * ) pcTextForTask2, 2, NULL );
```

○ 실행 결과

- Task 2는 더 높은 Priority이고 항상 실행 가능하므로 계속 Running 상태가 된다.
- Task 1은 CPU 시간을 받지 못해 Starvation 상태에 빠진다.

○ 핵심 원인

- Null Loop 와 출력만 반복하면서 Blocked 상태에 들어가지 않기 때문이다.
- 높은 Priority의 연속 처리 Task 가 낮은 Priority Task 를 완전히 굶길 수 있다.

Task 관리

☐ Event-Driven Task

- 처리할 Event가 없을 때 Blocked 상태로 들어가 CPU를 사용하지 않는다.

상태 또는 Event	설명	대표 예
Temporal Event	지연 시간 만료 또는 절대 시간 도달처럼 시간에 의해 발생	10ms 가 지날 때까지 대기
Synchronization Event	다른 Task 또는 Interrupt에서 발생	Queue 데이터 도착
Timeout	두 조건 중 먼저 만족	데이터 도착 또는 대기 시간 만료

☐ Not Running의 하위 상태

- Ready : 실행할 준비가 되었지만 현재 CPU를 사용하지 않는다.
- Blocked : 시간 또는 동기화 Event를 기다린다.
- Suspended
 - vTaskSuspend()로 진입한다.
 - vTaskResume() 또는 xTaskResumeFromISR()로 해제한다.

Task 관리

☐ Synchronization Event

- Queue, Binary Semaphore, Counting Semaphore, Mutex, Recursive Mutex를 통해 전달할 수 있다.
- Event Group, Stream Buffer, Message Buffer, Direct-to-Task Notification도 Synchronization Event를 만들 수 있다.

☐ Timeout

- Synchronization Event와 최대 대기 시간을 함께 지정할 수 있다.
- Queue Data를 10 ms 대기하면 Data 도착과 10 ms 경과 중 먼저 만족된 조건으로 Blocked 상태를 빠져나온다.

☐ Suspended 상태와 Ready 상태

- Suspended Task는 Resume API가 호출될 때까지 Scheduler가 선택할 수 없다.
- Ready Task는 실행 준비가 되었지만 현재 CPU를 사용하지 않는다.

Task 관리

☐ 상태 정의

- Suspended도 Not Running의 하위 상태이다.
- Suspended 상태의 Task는 Scheduler가 선택할 수 없다.

☐ 상태 전환

- vTaskSuspend()를 호출하면 Suspended 상태에 들어간다.
- vTaskResume() 또는 xTaskResumeFromISR()로만 빠져나올 수 있다.

☐ 사용 빈도

- 대부분의 애플리케이션은 Suspended 상태를 사용하지 않는다.

Task 관리

□ 상태 정의

- Blocked 상태도 아니고 Suspended 상태도 아닌 Not Running Task이다.
- 실행할 수 있고 실행할 준비가 되었지만 현재 CPU를 사용하지 않는다.

□ Scheduler의 선택

- Ready Task 중 Priority가 가장 높은 Task를 Running 상태로 전환한다.
- 같은 Priority의 Ready Task가 여러 개이면 설정된 Scheduling Algorithm에 따라 실행 기회를 나눈다.

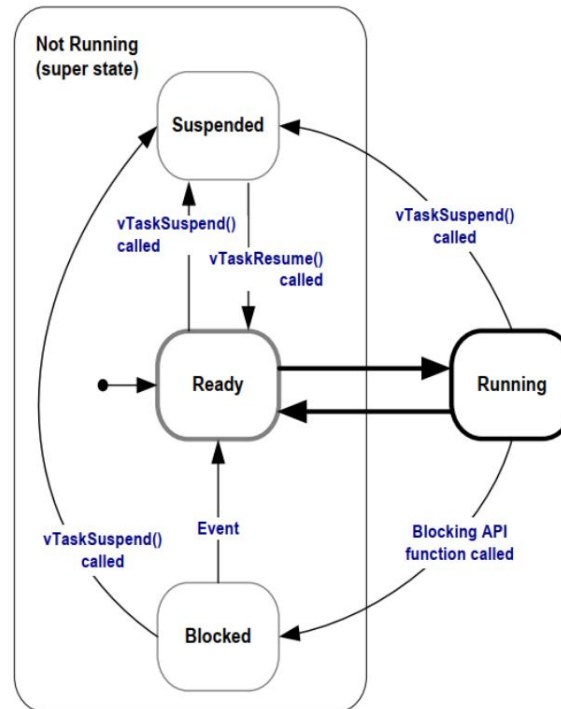
Task 관리

□ 상태와 전이 API

현재 상태	전이 조건	다음 상태
Ready	Scheduler가 선택	Running
Running	Scheduler가 다른 Task를 선택	Ready
Running	시간 또는 동기화 Event 대기	Blocked
Blocked	Event 발생 또는 Timeout 만료	Ready
Running 또는 Ready	vTaskSuspend() 호출	Suspended
Suspended	vTaskResume() 계열 호출	Ready

Task 관리

□ Ready, Running, Blocked, Suspended 상태



task state machine

Figure 4.7 Full

- Ready → Running : Scheduler가 실행할 Task를 선택한다.
- Running → Blocked : Blocking API를 호출한다.
- Blocked → Ready : 기다리던 Event가 발생한다.

Task 관리

□ vTaskDelay() API 함수

- 호출 시점부터 지정한 Tick 수 동안 호출 Task를 Blocked 상태로 둔다.

```
void vTaskDelay( TickType_t xTicksToDelay );
```

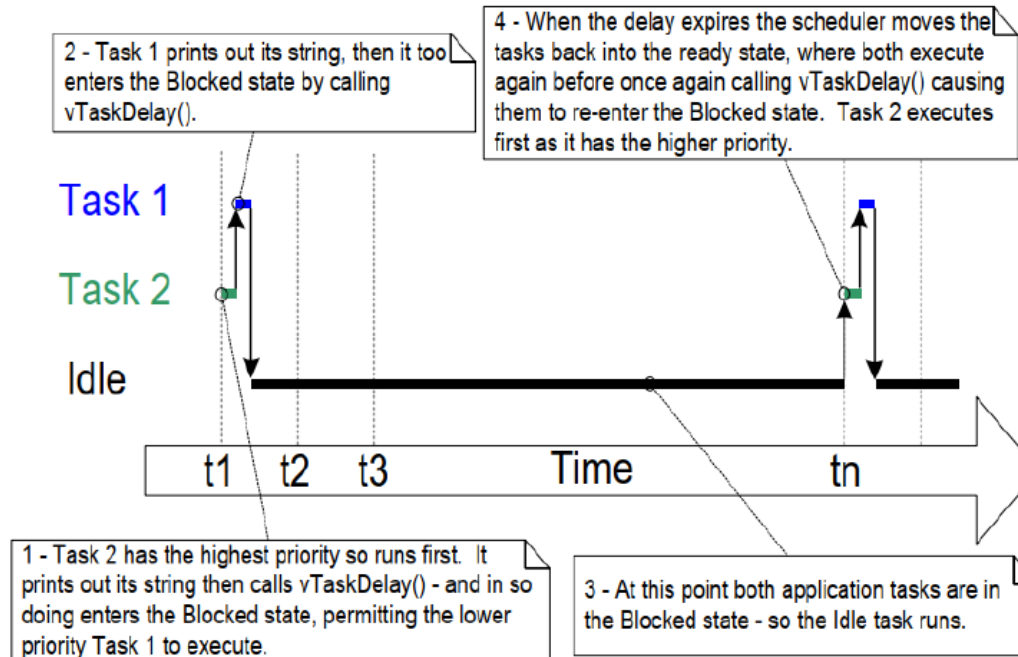
```
void vTaskFunction( void * pvParameters )
{
    char * pcTaskName = ( char * ) pvParameters;
    const TickType_t xDelay250ms = pdMS_TO_TICKS( 250 );

    for( ;; )
    {
        vPrintLine( pcTaskName );
        vTaskDelay( xDelay250ms );
    }
}
```

- INCLUDE_vTaskDelay=1이어야 사용할 수 있다. Tick Count 10,000에서 vTaskDelay(100)을 호출하면 10,100이 될 때까지 Blocked 상태에 머문다.
- xTicksToDelay는 호출 Task가 Ready 상태로 돌아오기 전까지 Blocked 상태에 머무를 Tick 수를 지정한다.pdMS_TO_TICKS()를 사용하면 밀리초를 Tick 수로 변환할 수 있다.
- vTaskDelay(pdMS_TO_TICKS(100))은 약 100ms 동안 Task를 Blocked 상태로 둔다.

Task 관리

- Null Loop를 `vTaskDelay()`로 교체
 - 지연 중에는 CPU 시간을 사용하지 않는다.



- 높은 Priority Task가 Blocked 상태에 있는 동안 낮은 Priority Task가 Running 상태가 될 수 있다.
- 두 Task가 모두 Blocked 상태이면 Idle Task가 실행된다.
- `vTaskDelay()` 사용 시 Task는 실제 작업을 수행할 때만 CPU를 사용하며, Idle Task의 실행 시간은 시스템의 여유 처리 능력을 나타낸다.

Task 관리

□ 고정 주기의 주기적 Task

- `vTaskDelay()`의 지연 종료 시점은 함수를 호출한 시점을 기준으로 한 상대 시간이다.
 - 매 주기마다 정확히 같은 주기로 실행해야 하면 호출 시점의 흔들림이 누적될 수 있다.
- `vTaskDelayUntil()`은 Ready 상태로 전환할 정확한 Tick Count를 계산하므로 고정 주기 Task에 적합하다.

□ 매개변수

- `pxPreviousWakeTime`
 - Task가 직전에 Blocked 상태를 빠져나온 시각을 보관한다.
 - 첫 사용 전 현재 Tick Count로 초기화하며, 이후에는 함수가 자동으로 갱신한다.
- `xTimeIncrement`
 - 주기 길이를 Tick 단위로 지정한다.
 - `pdMS_TO_TICKS()`로 밀리초를 Tick으로 변환할 수 있다.

Task 관리

- 기준 시각을 누적해서 관리하므로 일정한 실행 주기를 유지할 수 있다.

```
void vTaskDelayUntil(  
    TickType_t * pxPreviousWakeTime,  
    TickType_t xTimeIncrement  
);
```

```
void vTaskFunction( void * pvParameters )  
{  
    char * pcTaskName = ( char * ) pvParameters;  
    TickType_t xLastWakeTime;  
    xLastWakeTime = xTaskGetTickCount();  
    for( ;; )  
    {  
        vPrintLine( pcTaskName );  
        vTaskDelayUntil( &xLastWakeTime, pdMS_TO_TICKS( 250 ) );  
    }  
}
```

Task 관리

- 두 개의 Continuous Processing Task와 하나의 Periodic Task를 함께 실행
 - Priority 1의 Continuous Processing Task는 Blocked API를 호출하지 않아 항상 Ready 또는 Running 상태이다.

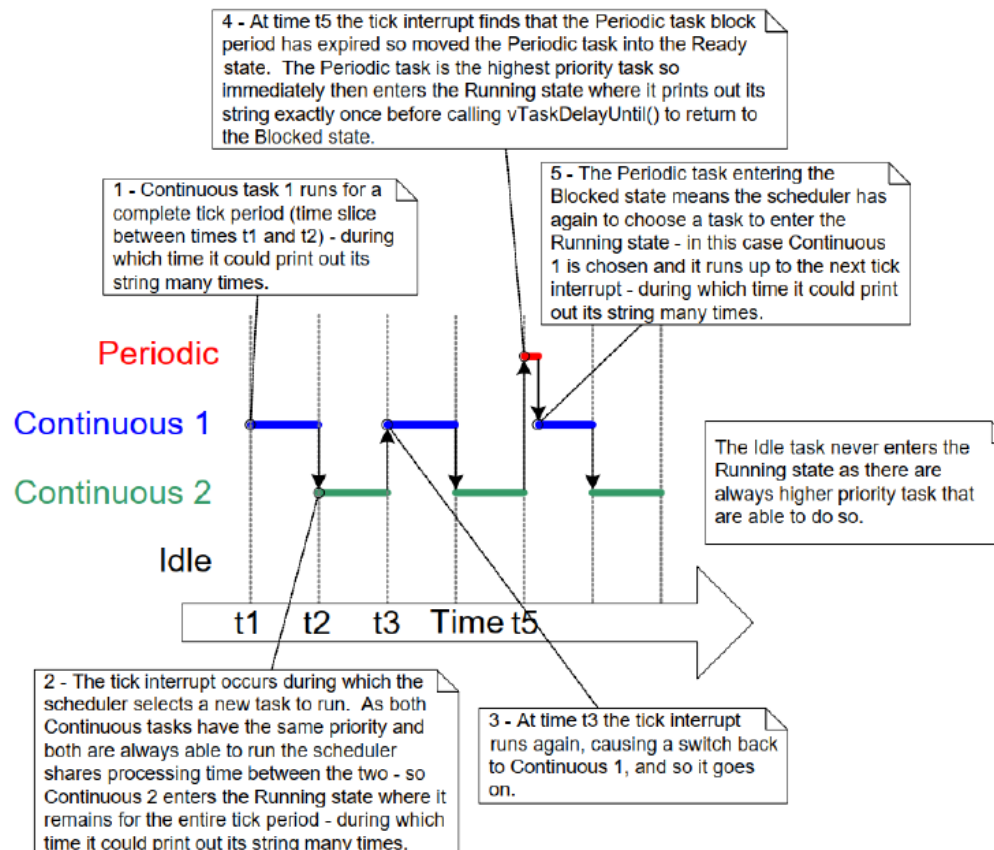


Figure 4.12 The execution pattern of Example 4.6

Task 관리

□ Periodic Task는 출력 후 vTaskDelayUntil()을 호출하여 다음 주기까지, Blocked 상태로 들어간다.

```
void vContinuousProcessingTask( void *  
pvParameters )  
{  
    char * pcTaskName = ( char * )  
pvParameters;  
    for( ;; )  
    {  
        vPrintLine( pcTaskName );  
    }  
}
```

```
void vPeriodicTask( void * pvParameters )  
{  
    TickType_t xLastWakeTime =  
xTaskGetTickCount();  
    const TickType_t xDelay3ms = pdMS_TO_TICKS(  
3 );  
    for( ;; )  
    {  
        vPrintLine( "Periodic task is running" );  
        vTaskDelayUntil( &xLastWakeTime, xDelay3ms  
);  
    }  
}
```

□ 높은 Priority의 Periodic Task가 Ready 상태가 되는 순간 낮은 Priority의 Continuous Task를 Preempt한다.

- Periodic Task가 다시 Blocked 상태로 들어가면
- 두 Continuous Task가 동일 Priority에서 CPU를 공유한다.

Task 관리

□ Scheduler는 vTaskStartScheduler()가 호출될 때 Idle Task를 자동으로 생성한다.

- 시스템에는 항상 Running 상태에 들어갈 수 있는 Task가 하나 이상 있어야 한다.
- Idle Task는 거의 단순 Loop만 수행하므로 항상 실행 가능하다.
- Priority는 가능한 최저값인 0이다.
- 더 높은 Priority Task가 Ready 상태가 되면 즉시 Running 상태를 내어준다.

□ configIDLE_SHOULD_YIELD

- Priority 0의 Application Task와 CPU 시간을 공유하는 방식을 조절한다.

□ Task 삭제와 Idle Task

- 자기 자신을 삭제한 Task의 Kernel Resource는 Idle Task가 정리한다.
- 따라서 이런 시스템에서는 Idle Task를 Starvation시켜서는 안 된다.

Task 관리

□ Idle Hook 또는 Idle Callback은 Idle Task의 Loop가 한 번 반복될 때마다 자동으로 호출되는 Application 함수이다.

○ Idle 시간에 낮은 Priority의 기능을 수행하도록 활용할 수 있다.

```
void vApplicationIdleHook( void );
```

○ 대표적인 활용 방법

- 별도의 Application Task 없이 Background 또는 Continuous Processing 기능을 실행한다.
- Idle Task가 실행된 시간을 측정하여 시스템의 남은 처리 능력을 추정한다.
- Application 작업이 없을 때 Processor를 Low-Power Mode로 전환한다.

Task 관리

□ 4.8.2 Idle Task Hook 구현 시 제한 사항

- Idle Hook은 자신을 Block 또는 Suspend해서는 안 된다.
- 자기 삭제 Task를 사용하면 적절한 시간 안에 반환해야 한다.

```
volatile unsigned long ulIdleCycleCount = 0UL;

void vApplicationIdleHook( void )
{
    ulIdleCycleCount++;
}
```

- 예제 4.7은 Idle Hook에서 전역 Counter를 증가시킨다.
- Counter의 증가량을 통해 Idle 시간이 얼마나 많은지 관찰할 수 있다.

Task 관리

□ Idle Hook이 실제로 호출되게 하려면 configUSE_IDLE_HOOK을 1로 설정한다.

○ 주기 Task에서 ulIdleCycleCount를 함께 출력한다.

```
void vTaskFunction( void * pvParameters )
{
    char * pcTaskName = ( char * ) pvParameters;
    const TickType_t xDelay250ms = pdMS_TO_TICKS( 250 );

    for( ;; )
    {
        vPrintLineAndNumber( pcTaskName, ulIdleCycleCount );
        vTaskDelay( xDelay250ms );
    }
}
```

□ 원문의 Windows 예제에서는 Application Task가 한 번 실행하는 사이 Idle Hook이 약 수백만 번 실행된다.

○ 정확한 횟수는 Hardware 성능에 따라 달라진다.

Task 관리

□ vTaskPrioritySet() API 함수와 uxTaskPriorityGet() API 함수

```
void vTaskPrioritySet(  
    TaskHandle_t xTask,  
    UBaseType_t uxNewPriority  
);  
  
UBaseType_t uxTaskPriorityGet(  
    TaskHandle_t xTask  
);
```

- vTaskPrioritySet()은 Scheduler가 시작된 이후 Task의 Priority를 변경한다.
- xTask에 NULL을 전달하면 호출 Task 자신의 Priority를 변경한다.
- uxNewPriority가 configMAX_PRIORITIES-1보다 크면 최대값으로 제한한다.
- INCLUDE_vTaskPrioritySet을 1로 설정해야 사용할 수 있다.
- uxTaskPriorityGet()은 특정 Task의 현재 Priority를 반환한다.
- INCLUDE_uxTaskPriorityGet을 1로 설정해야 사용할 수 있으며, NULL은 호출 Task 자신을 뜻한다.

Task 관리

□ Task 1과 Task 2 구현

```
TaskHandle_t xTask2Handle = NULL;

void vTask1( void * pvParameters )
{
    UBaseType_t uxPriority = uxTaskPriorityGet( NULL );

    for( ;; )
    {
        vPrintLine( "Task 1 is running" );
        vPrintLine( "About to raise the Task 2 priority" );
        vTaskPrioritySet( xTask2Handle, uxPriority + 1 );
    }
}
```

```
void vTask2( void * pvParameters )
{
    UBaseType_t uxPriority = uxTaskPriorityGet( NULL );

    for( ;; )
    {
        vPrintLine( "Task 2 is running" );
        vPrintLine( "About to lower the Task 2 priority" );
        vTaskPrioritySet( NULL, uxPriority - 2 );
    }
}
```

- Scheduler는 항상 가장 높은 Ready Task를 즉시 Running 상태로 선택한다.

Task 관리

- Task 2 Handle을 생성 시 저장한다.

```
TaskHandle_t xTask2Handle = NULL;

int main( void )
{
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 2, NULL );

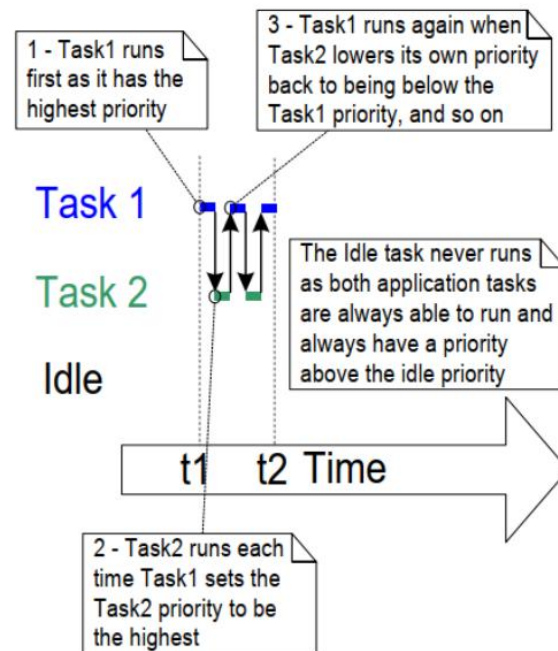
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, &xTask2Handle );

    vTaskStartScheduler();
    for( ;; ) { }
}
```

- 자기 자신의 Priority를 조회하거나 변경할 때에는 NULL Handle을 사용할 수 있다.
- 다른 Task를 제어할 때에는 생성 시 얻은 Task Handle이 필요하다.

Task 관리

- 두 Task의 상대 Priority를 실행 중에 바꾸며 Scheduler의 동작을 확인한다.
- Task 1이 Task 2의 Priority를 자기보다 높게 올린다.



task execution when running Example 4.8

Figure 4.14 The sequence of

- Task 2가 즉시 Running 상태가 되어 Task 1을 Preempt한다.
- Task 2가 자신의 Priority를 낮추면 Task 1이 즉시 Running 상태로 돌아온다.

Task 관리

□ vTaskDelete()는 Task를 삭제한다.

```
void vTaskDelete( TaskHandle_t xTaskToDelete );
```

- FreeRTOSConfig.h에서 INCLUDE_vTaskDelete를 1로 설정해야 사용할 수 있다.
- 삭제한 Task는 더 이상 존재하지 않으며 다시 Running 상태가 될 수 없다.
 - xTaskToDelete에 삭제할 Task Handle을 전달하며, NULL은 호출 Task 자신을 뜻한다.
- 자기 삭제 Task의 TCB와 Stack 등 Kernel Memory는 Idle Task가 해제한다.
- Task 코드에서 별도로 할당한 Memory와 Resource는 개발자가 직접 해제해야 한다.
- Task를 계속 생성하고 삭제하는 설계보다 기존 Task를 재사용하는 방법을 우선 고려한다.

Task 관리

- Task 2의 생성·자기 삭제와 Idle Task의 Memory 정리 과정을 반복한다.
 - main()이 Priority 1의 Task 1을 생성한다.
 - Task 1이 Priority 2의 Task 2를 생성하면 Task 2가 즉시 실행한다.
 - Task 2는 자기 Handle을 사용하여 스스로 삭제한다.
 - Task 1은 다시 실행되어 100 ms 동안 vTaskDelay()로 Blocked 상태에 들어간다.
 - 그동안 Idle Task가 삭제된 Task 2의 Kernel Memory를 정리한다.
 - Task 1이 Ready 상태가 되면 Idle Task를 Preempt하고 Task 2를 다시 생성한다.

Task 관리

- main()은 Task 1만 생성하고 Scheduler를 시작한다.

```
TaskHandle_t xTask2Handle = NULL;

int main( void )
{
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 1, NULL );

    vTaskStartScheduler();
    for( ;; ) { }
}
```

- Task 1이 Priority 2의 Task 2를 생성한다.
 - 더 높은 Priority의 Task 2가 즉시 실행한다.
 - Task 2는 자기 Handle을 사용하여 스스로 삭제한다.

Task 관리

□ Task 1은 Task 2를 생성한 뒤 지연하고, Task 2는 자기 자신을 삭제한다.

```
void vTask1( void * pvParameters )
{
    const TickType_t xDelay100ms = pdMS_TO_TICKS( 100UL );
    for( ;; )
    {
        vPrintLine( "Task 1 is running" );
        xTaskCreate( vTask2, "Task 2", 1000, NULL, 2,
&xTask2Handle );
        vTaskDelay( xDelay100ms );
    }
}
```

```
void vTask2( void * pvParameters )
{
    vPrintLine(
        "Task 2 is running and about
to delete itself" );
    vTaskDelete( xTask2Handle );
}
```

□ 동적 메모리로 생성한 Task가 자기 자신을 삭제하면

- Idle Task가 TCB와 Stack 등 Kernel이 할당한 Memory를 해제한다.
- Task 코드에서 별도로 할당한 Memory나 Resource는 개발자가 직접 해제한다.

Task 관리

- Task 1이 Task 2를 만들고, Task 2가 자기 자신을 삭제한다.

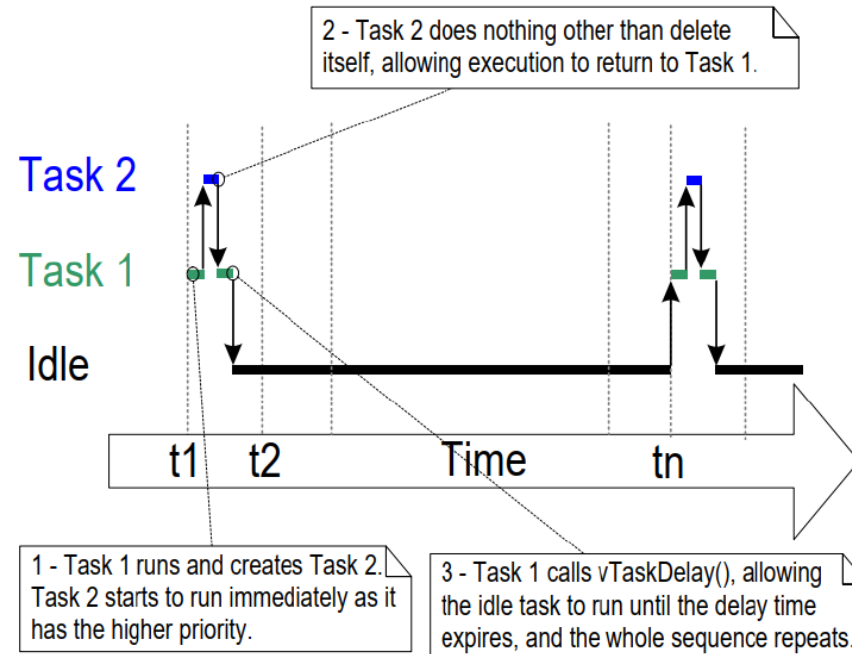


Figure 4.17 The execution sequence for Example 4.9

- Task 1이 `vTaskDelay()`로 Blocked 상태에 들어간다.
- 그동안 Idle Task가 삭제된 Task 2의 Kernel Memory를 정리한다.

Task 관리

□ Thread Local Storage(TLS)는 각 Task의 TCB 안에 Application이 임의의 데이터를 저장할 수 있게 한다.

- Reentrant하지 않은 함수가 전역 변수에 저장하던 상태를 Task별로 분리할 때 자주 사용한다.

설정 또는 API	역할
configUSE_NEWLIB_REENTRANT	newlib의 Task별 Reentrancy 지원
configUSE_PICOLIBC_TLS	picolibc의 Task별 TLS 지원
configUSE_C_RUNTIME_TLS_SUPPORT	사용자 정의 C Runtime TLS 활성화
configNUM_THREAD_LOCAL_STORAGE_POINTERS	Application TLS Pointer 배열 크기
pvTaskGet... / vTaskSet...	Application TLS Pointer 조회와 설정

□ 대표적인 예는 errno이다.

- strtod(), strtol() 같은 함수는 상세 오류 상태를 errno에 기록한다.
- TLS는 과도한 Critical Section 의존을 줄이는 데 적합하다.

Task 관리

□ Reentrant Function

- 여러 Thread 또는 Task에서 동시에 호출해도 부작용 없이 안전하게 동작한다.
- Non-Reentrant 함수를 TLS 없이 사용하면 공유 상태 보호를 위해 Critical Section이 필요할 수 있다.

라이브러리	FreeRTOSConfig.h 설정
newlib	configUSE_NEWLIB_REENTRANT
picolibc	configUSE_PICOLIBC_TLS

□ 대표적인 예는 ISO C 표준 라이브러리와 POSIX 시스템이 사용하는 errno이다.

- strtod(), strtol() 같은 함수는 상세 오류 상태를 errno에 기록한다.
- 멀티스레드 환경에서는 Task 또는 Thread별 errno가 필요하다.

Task 관리

□ Application이 자체 C Runtime TLS를 구현하려면 FreeRTOSConfig.h에 다음 매크로를 정의한다.

매크로	역할
configUSE_C_RUNTIME_TLS_SUPPORT	C Runtime TLS 지원 활성화
configTLS_BLOCK_TYPE	TLS Data를 저장할 C 자료형 정의
configINIT_TLS_BLOCK	TLS Block 초기화
configSET_TLS_BLOCK	Task Switch-In 시 TLS Block 설정
configDEINIT_TLS_BLOCK	TLS Block 해제 또는 종료 처리

Task 관리

□ C Runtime TLS와 별도로 Application 전용 Pointer 배열을 각 TCB에 포함할 수 있다.

- configNUM_THREAD_LOCAL_STORAGE_POINTERS를 0보다 큰 값으로 설정하면 활성화한다.

```
void * pvTaskGetThreadLocalStoragePointer(  
    TaskHandle_t xTaskToQuery,  
    BaseType_t xIndex  
);  
  
void vTaskSetThreadLocalStoragePointer(  
    TaskHandle_t xTaskToSet,  
    BaseType_t xIndex,  
    void * pvValue  
);
```

Task 관리

☐ Running

- 실제로 CPU 시간을 사용하고 있는 Task이다.
- 단일 Core에서는 한 시점에 하나의 Task만 Running 상태가 될 수 있다.

☐ Ready

- Blocked 또는 Suspended 상태가 아니며 실행 준비가 된 Task이다.
- Scheduler는 Ready Task 중 가장 높은 Priority의 Task를 Running 상태로 선택한다.

☐ Blocked 상태와 Event

- Blocked Task는 Event를 기다리며, Event가 발생하면 자동으로 Ready 상태로 이동한다.
- Temporal Event는 자연 시간 만료와 같은 시간 기반 사건이다.
- Synchronization Event는 Queue, Semaphore, Task Notification, Event Group 등을 통해 전달된다.
- FreeRTOS는 AMP와 SMP도 지원하지만 이 절은 단일 Core를 기준으로 한다.

Task 관리

□ Scheduling Algorithm

- Ready 상태 Task 중 어느 Task를 Running 상태로 전환할지 결정하는 Software 규칙이다.

Scheduling Algorithm	Prioritized	configUSE_PREEMPTION	configUSE_TIME_SLICING
Preemptive + Time Slicing	Yes	1	1
Preemptive without Time Slicing	Yes	1	0
Cooperative	No	0	Any

□ 모든 단일 Core 설정에서 같은 Priority를 가진 Ready Task는 차례로 실행 기회를 얻는다.

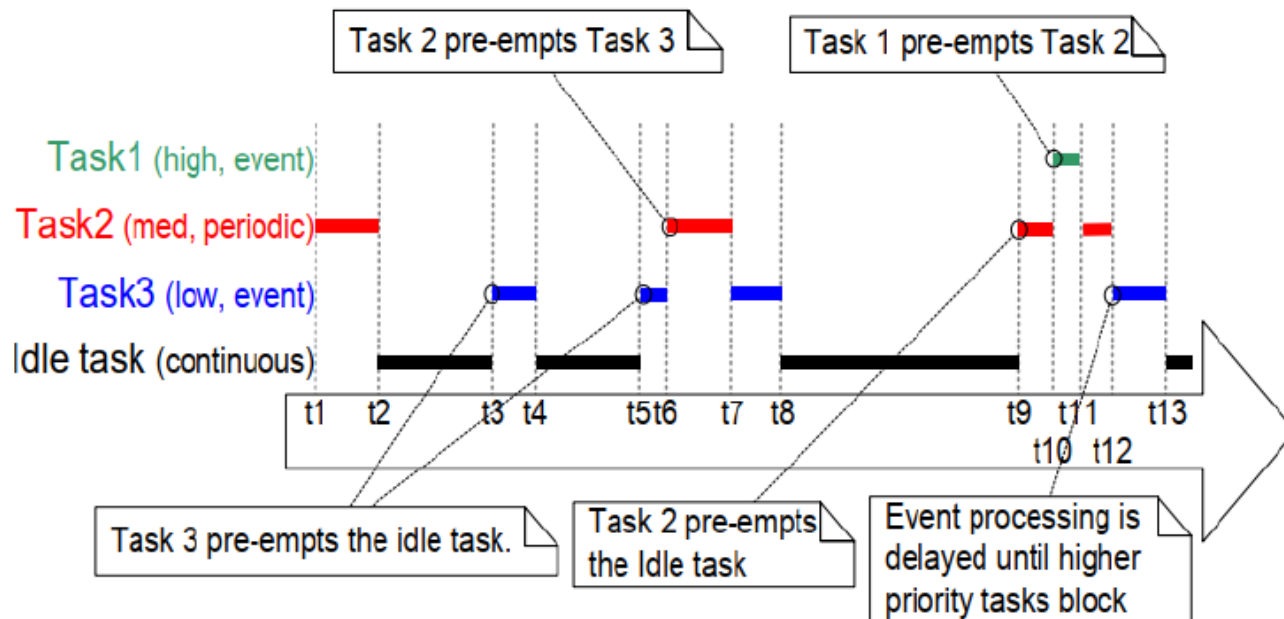
- 이를 흔히 Round Robin Scheduling이라고 한다.
- Round Robin은 정확히 같은 CPU 시간을 보장하지 않고 실행 순서를 번갈아 준다는 의미이다.
- configUSE_TICKLESS_IDLE도 Scheduling에 영향을 주지만 이 절에서는 0으로 가정한다.

Task 관리

- ☐ configUSE_PREEMPTION=1, configUSE_TIME_SLICING=1
 - Fixed Priority
 - Scheduler가 Task에 할당된 Priority를 자동으로 바꾸지 않는다.
- ☐ Task 자체는 API로 자신이나 다른 Task의 Priority를 변경할 수 있다.
 - Preemptive
 - 더 높은 Priority Task가 Ready 상태가 되면 현재 Task를 즉시 Preempt하며, Tick 시점이 아니어도 발생할 수 있다.
- ☐ Time Slicing
 - 같은 Priority의 Ready Task가 여러 개이면 Tick마다 새 Task를 선택한다.
 - 한 Time Slice는 두 Tick Interrupt 사이의 시간과 같다.

Task 관리

□ Event 발생으로 Ready 상태가 된 높은 Priority Task가 현재 Running Task를 즉시 Preempt한다.



- Task 1 : 시스템에서 가장 높은 Priority의 Event-Driven Task
- Task 2 : Task 3보다 높고 Task 1보다 낮은 Priority의 Periodic Task
- Task 3은 낮은 Priority의 Event-Driven Task이고, Idle Task는 최저 Priority이다.

Task 관리

- Idle Task와 Task 2는 Priority 0의 Continuous Processing Task로 Tick마다 CPU 시간을 번갈아 사용한다.
- 높은 Priority의 Task 1은 Event가 발생하면 즉시 Preempt한다.

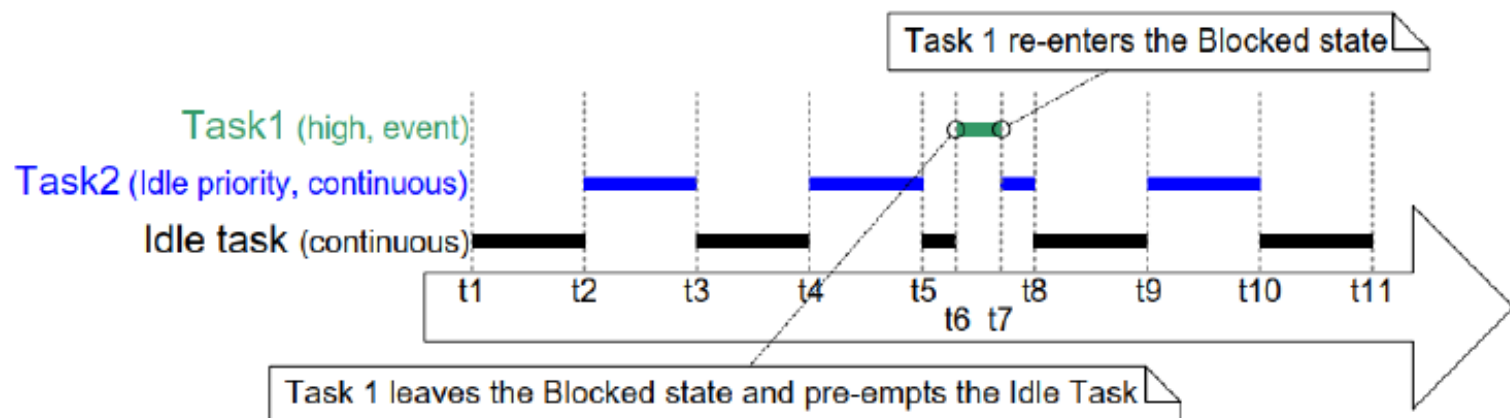
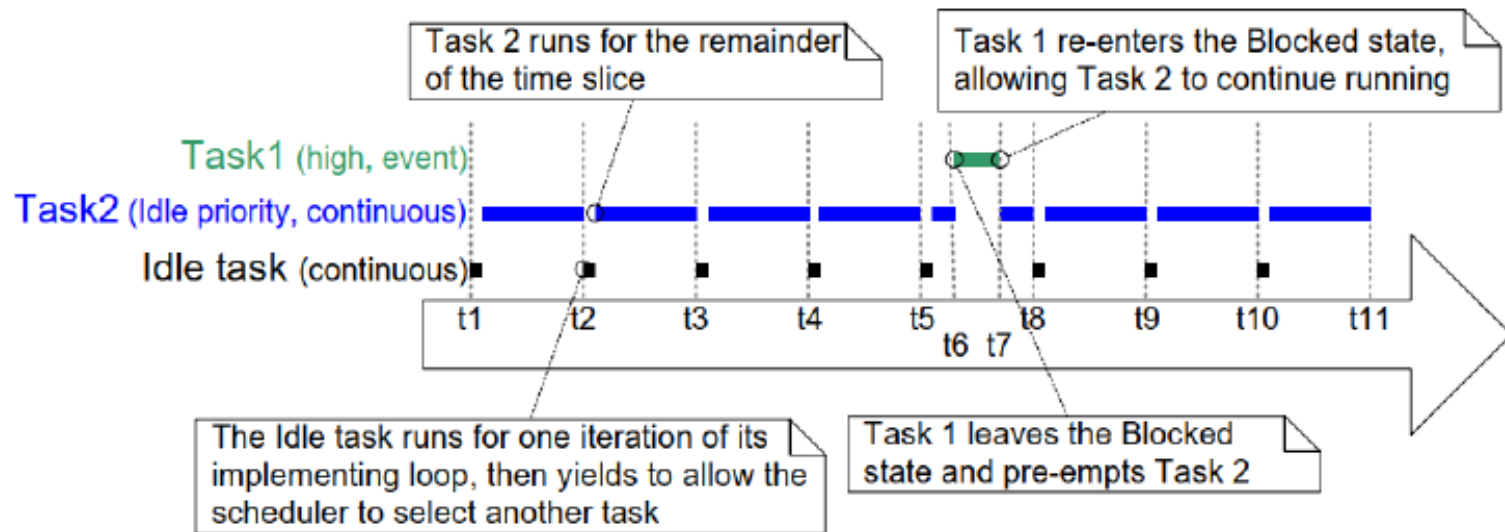


Figure 4.19 Execution pattern highlighting task prioritization and time slicing in a hypothetical application in which two tasks run at the same priority

- configIDLE_SHOULD_YIELD=0이면 Idle Task가 자기 Time Slice 전체를 사용한다.

Task 관리

- configIDLE_SHOULD_YIELD=1이면 같은 Priority 0의 Ready Task가 있을 때 Idle Task가 남은 Time Slice를 양보한다.



- 그 다음 Task는 남은 Time Slice만 실행할 수 있다.

Task 관리

□ configUSE_PREEMPTION=1, configUSE_TIME_SLICING=0

○ 같은 Priority Task 사이에서 Tick마다 강제로 교체하지 않는다.

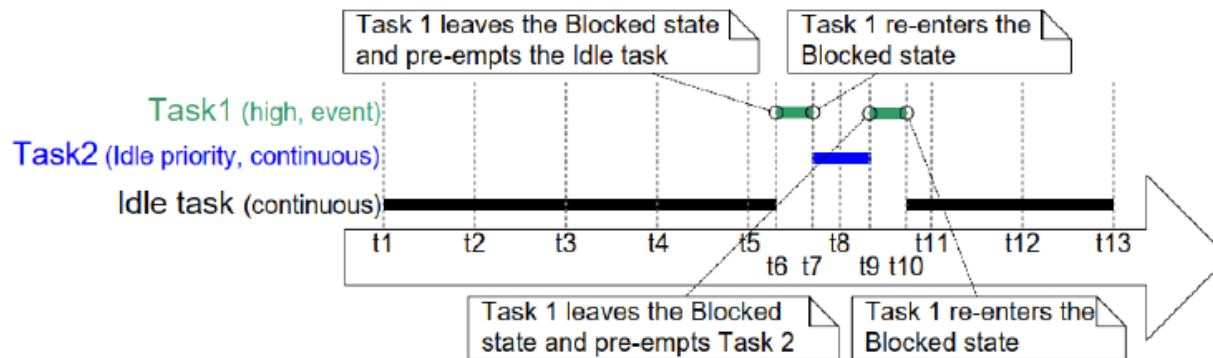


Figure 4.21 Execution pattern that demonstrates how tasks of equal priority can receive hugely different amounts of processing time when time slicing is not used

- Scheduler가 새 Task를 선택하는 경우
 - 더 높은 Priority의 Task가 Ready 상태가 된다.
 - 현재 Running Task가 Blocked 또는 Suspended 상태로 들어간다.
- Context Switch 횟수가 줄어 Scheduler Overhead가 감소한다.
- 같은 Priority Task가 서로 다른 CPU 시간을 받을 수 있어 신중하게 사용해야 한다.

Task 관리

□ configUSE_PREEMPTION=0

○ Cooperative Scheduler에서는 Task가 자동으로 Preempt되지 않는다.

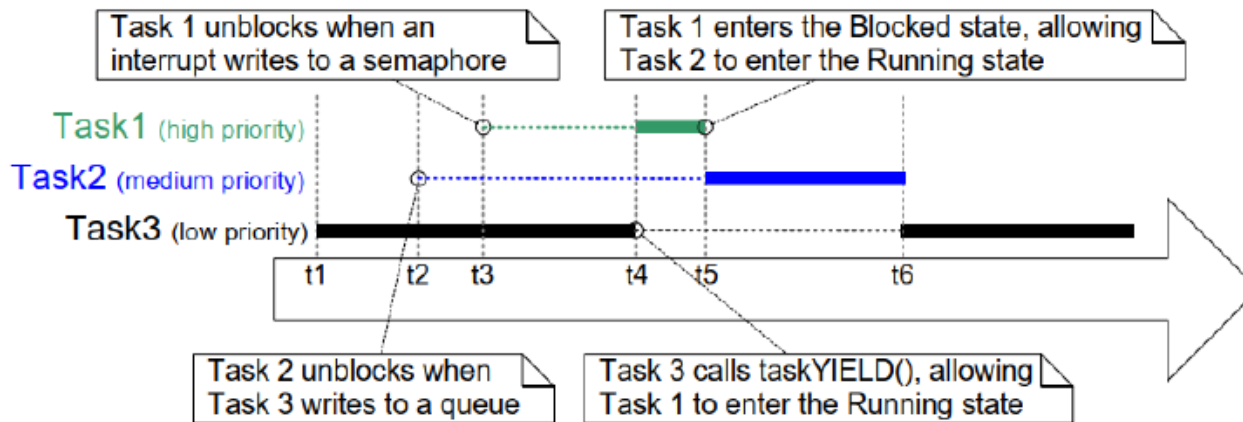


Figure 4.22 Execution pattern demonstrating the behavior of the cooperative scheduler

○ Application ISR이 명시적으로 Context Switch를 요청하지 않는다고 가정한다.

➤ Running Task가 Blocked 상태로 들어갈 때 Context Switch가 발생한다.

➤ Running Task가 taskYIELD()를 호출할 때 Context Switch가 발생한다.

○ Task가 자동으로 Preempt되지 않으므로 Time Slicing을 사용할 수 없다.

○ 높은 Priority Task도 현재 Task가 Block 또는 Yield할 때까지 기다려야 한다.

Task 관리

□ Task 1이 UART에 "abcdefghijklmnop"를 쓰기 시작해 "abcdefg"까지 출력한 뒤 Running 상태를 잃는다.

- Task 2가 Running 상태가 되어 "123456789"를 모두 출력한다.
- Task 1이 다시 Running 상태가 되어 나머지 "hijklmnop"를 출력한다.

```
Task 1 : abcdefg      hijklmnop
Task 2 :      123456789

UART   : abcdefg123456789hijklmnop
```

□ Preemptive Scheduler

- Running Task가 Resource를 일관되지 않은 상태로 둔 순간에도 Preempt될 수 있다.

□ Cooperative Scheduler

- Resource가 불완전한 상태일 때 Yield하지 않도록 설계하기 쉽다.
- 다만 현재 Task가 Block 또는 taskYIELD()를 호출할 때까지 높은 Priority Task의 실행이 늦어진다.

Task 관리

- ☐ Task는 Running / Ready / Blocked / Suspended 상태를 전환한다.
- ☐ Scheduler는 실행 가능한 가장 높은 Priority Task를 선택한다.
- ☐ vTaskDelay()는 상대 지연, vTaskDelayUntil()은 고정 주기 실행에 적합하다.
- ☐ Idle Task는 Priority 0으로 자동 생성되며 여유 CPU 사용과 삭제 Task
- ☐ Resource 정리를 담당한다.
- ☐ FreeRTOS는 Time Slicing을 사용하는 Preemptive, 사용하지 않는 Preemptive, Cooperative Scheduling을 지원한다.